Industrial Internship Report on

"Python URL Shortener using Flask and SQLite"

Prepared by

V Adhithiyaa

Executive Summary
This report provides details of the Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with Industrial Partner Uni Converge Technologies Pvt Ltd (UCT). This internship was focused on a project/problem statement provided by UCT. We had to finish the project including the report in 6 weeks' time. My project was the Python URL Shortener, a web-based application developed using Python, Flask, SQLite, HTML and CSS that converts long URLs into short, easy-to-share links. Users enter a valid URL and receive a unique short code; when the short link is opened, the application looks up the original URL in the database, increments a click counter and redirects the browser to the destination. This internship gave me a very good opportunity to get exposure to Industrial problems and design/implement solution for that. It was an overall great experience to have this internship.

TABLE OF CONTENTS

1 Preface 3
2 Introduction 5
2.1 About UniConverge Technologies Pvt Ltd 5
2.2 About upskill Campus 9
2.3 Objective 10
2.4 Reference 10
2.5 Glossary 10
3 Problem Statement 11
4 Existing and Proposed solution 12
5 Proposed Design/ Model 13
5.1 High Level Diagram (if applicable) 13
5.2 Low Level Diagram (if applicable) 14
5.3 Interfaces (if applicable) 14
6 Performance Test 15
6.1 Test Plan/ Test Cases 15
6.2 Test Procedure 16
6.3 Performance Outcome 16
7 My learnings 18
8 Future work scope 19

1 Preface

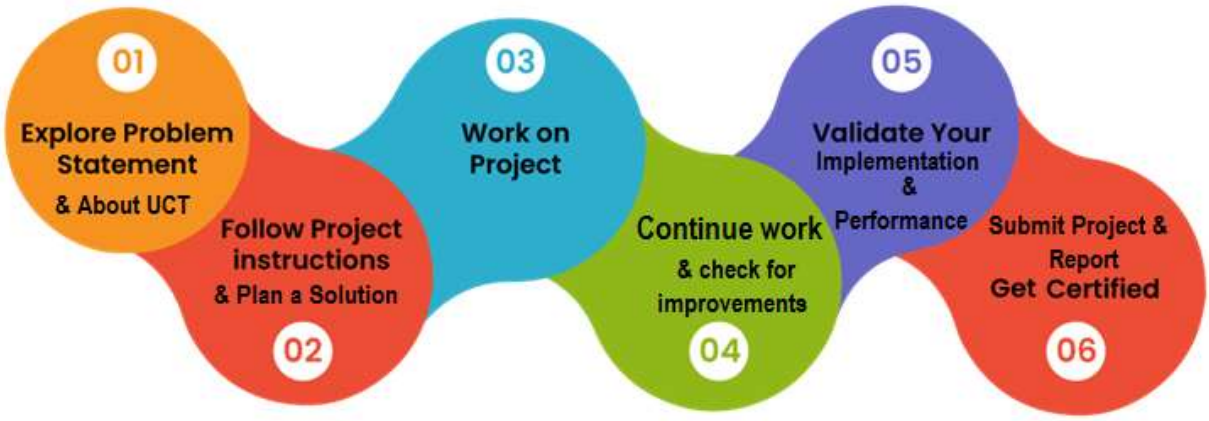
This report summarises the six weeks of work carried out during the Industrial Internship facilitated by upskill Campus (USC) and The IoT Academy in association with UniConverge Technologies Pvt Ltd (UCT). The internship was built around completing a working project, documenting it and submitting it within the given time.

A relevant internship is an important step in career development. It bridges the gap between classroom theory and industry practice, builds confidence with real tools and workflows, and gives practical, demonstrable experience that improves employability.

My project was the Python URL Shortener, a web application built with Python, Flask, Flask-SQLAlchemy, SQLite, HTML and CSS. It converts long web addresses into short links, stores the mapping between each short code and its original URL in a database, redirects visitors to the original website, and keeps a count of clicks.

USC and UCT gave me the opportunity to work on a real, end-to-end problem statement, along with the guidance, structure and timelines needed to deliver it as a working application with documentation and source code on GitHub.

The programme was planned in stages, from understanding the problem statement and setting up the environment, through design, implementation and testing, to documentation and final submission, as shown below.



Through this project I gained hands-on experience in web development with Flask, database operations with SQLAlchemy and SQLite, URL validation, unique identifier generation, HTTP redirects, click tracking, error handling and user interface design. I also learned to organise project files, manage dependencies, test functionality, write technical documentation and use GitHub for version control. Overall, it was a rewarding and practical experience.

I sincerely thank upskill Campus, The IoT Academy and UniConverge Technologies Pvt Ltd for this opportunity, and the mentors, trainers and coordinators who guided me throughout the internship [add their names here]. I also thank my family, friends and peers for their constant support and encouragement.

To my juniors and peers: start early, build something small that works end to end, and then improve it step by step. Test often, document as you go, and do not hesitate to ask for help. Consistent effort matters more than perfection.

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various **Cutting Edge Technologies e.g. Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end** etc.



i. UCT IoT Platform (uct Insight)

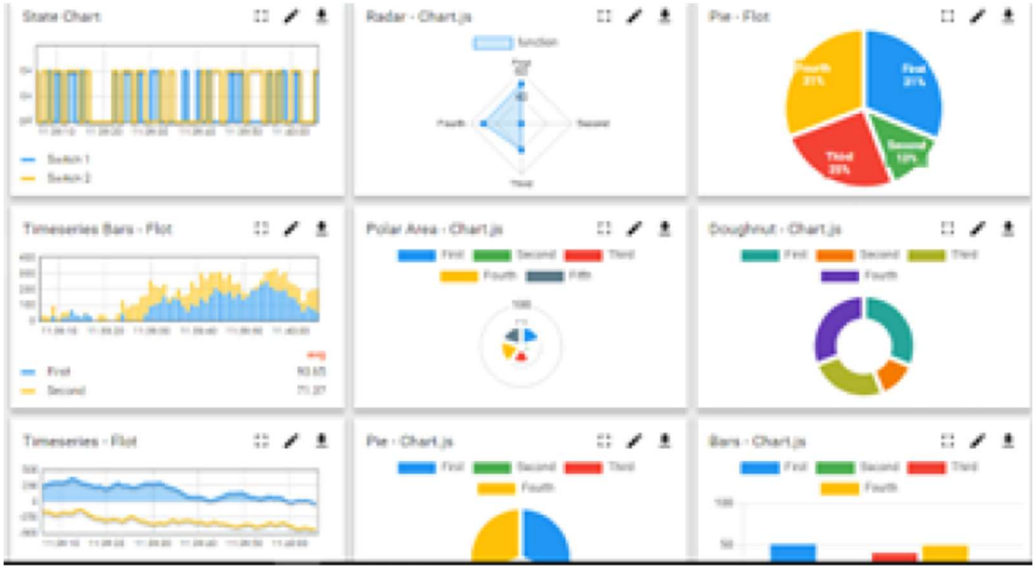
UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable "insight" for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

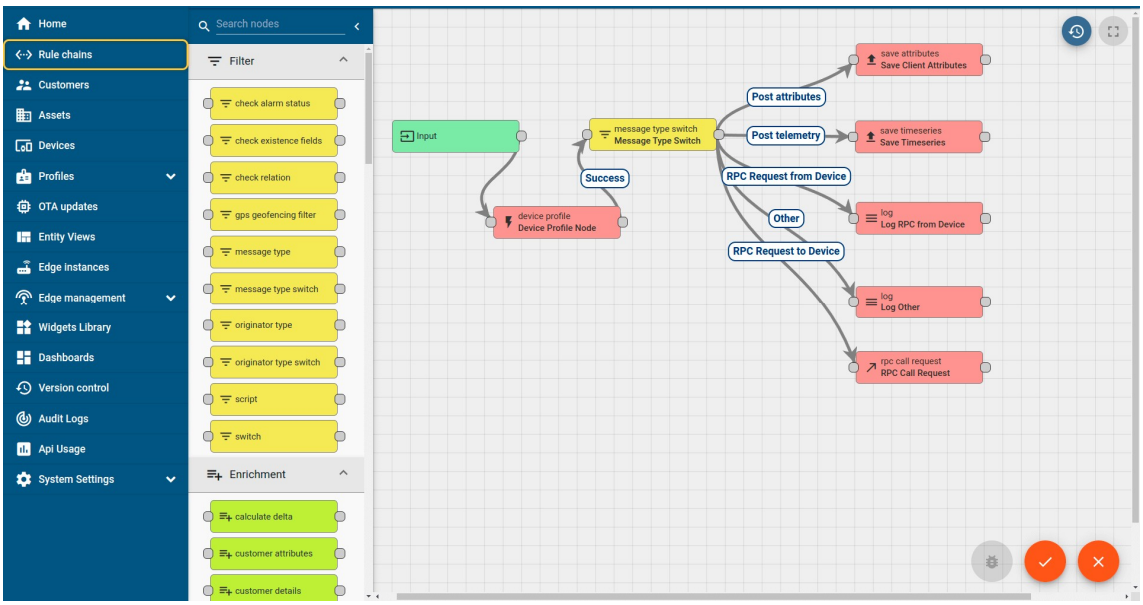
- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA

- It supports both cloud and on-premises deployments.

It has features to

- Build Your own dashboard
- Analytics and Reporting
- Alert and Notification
- Integration with third party application(Power BI, SAP, ERP)
- Rule Engine





ii. Smart Factory Platform (FACTORY WATCH)

Factory watch is a platform for smart factory needs.

It provides Users/ Factory

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleased the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they what to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.



Machine	Operator	Work Order ID	Job ID	Job Performance	Job Progress		Output		Rejection	Time (mins)				Job Status	End Customer
					Start Time	End Time	Planned	Actual		Setup	Pred	Downtime	Idle		
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i

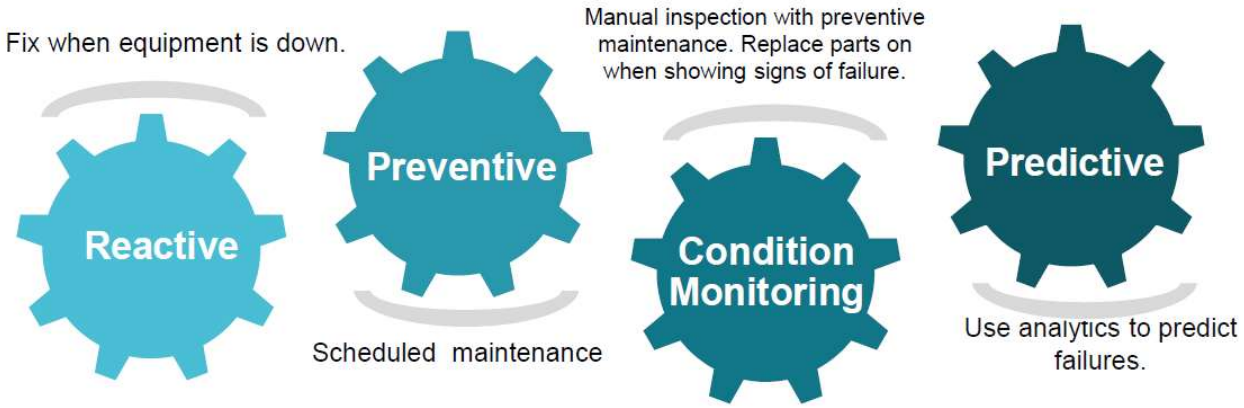


iii. LoRaWAN based Solution

UCT is one of the early adopters of LoRAWAN teschnology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

iv. Predictive Maintenance

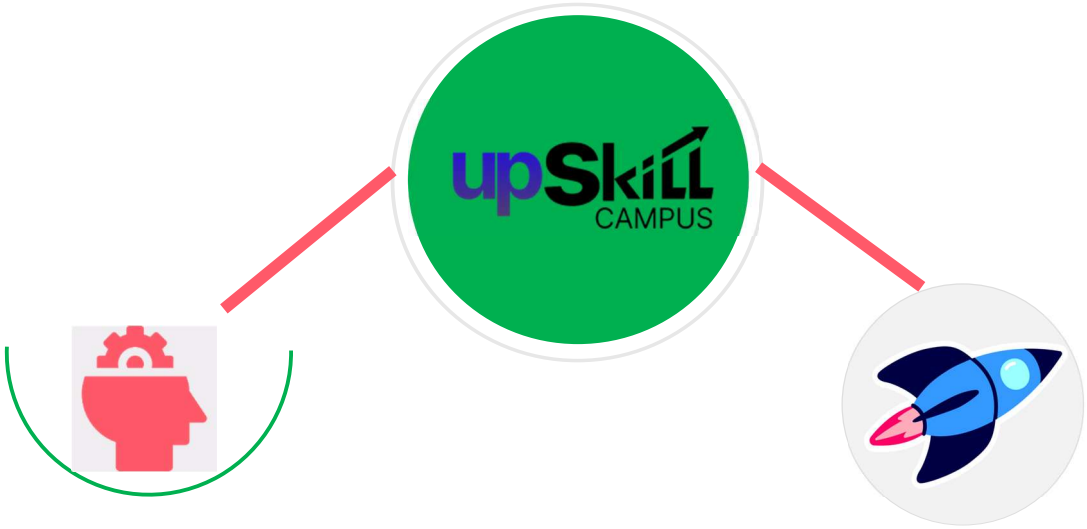
UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.



2.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

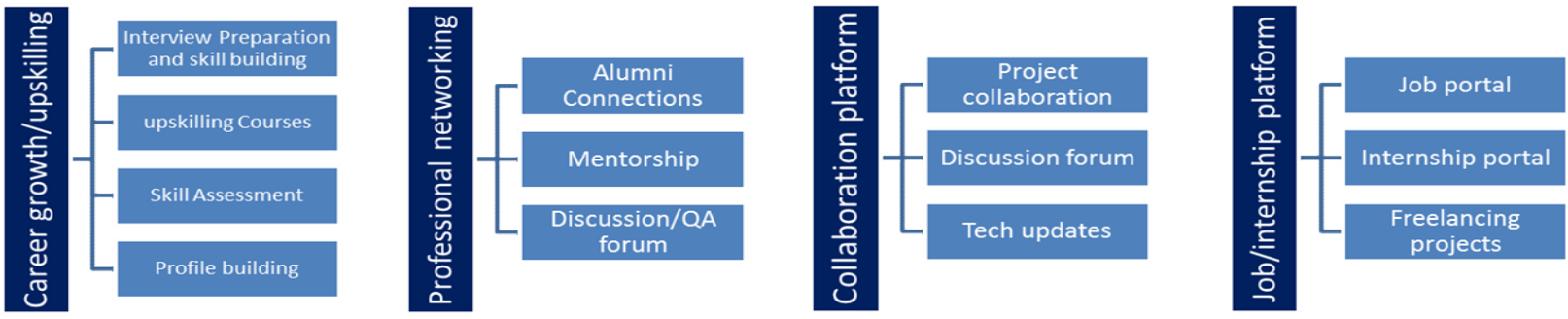
USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.



Seeing need of upskilling in self paced manner along-with additional support services e.g. Internship, projects, interaction with Industry experts, Career growth Services

upSkill Campus aiming to upskill 1 million learners in next 5 year

https://www.upskillcampus.com/



2.3 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

2.4 Objectives of this Internship program

The objective for this internship program was to

- ☛ get practical experience of working in the industry.
- ☛ to solve real world problems.
- ☛ to have improved job prospects.
- ☛ to have Improved understanding of our field and its applications.
- ☛ to have Personal growth like better communication and problem solving.

2.5 Reference

[1] Flask Documentation, Pallets Projects. https://flask.palletsprojects.com/

[2] Flask-SQLAlchemy Documentation. https://flask-sqlalchemy.palletsprojects.com/

[3] SQLite Documentation. https://www.sqlite.org/docs.html

[4] Python Documentation. https://docs.python.org/3/

[5] MDN Web Docs, HTTP Redirections. https://developer.mozilla.org/en-US/docs/Web/HTTP/Redirections

2.6 Glossary

Terms	Acronym
URL	Uniform Resource Locator
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
ORM	Object Relational Mapping
SQL	Structured Query Language

3 Problem Statement

Long web addresses are difficult to share, remember and type. They often contain deep paths, query strings and tracking parameters that break across lines in emails and chat messages, look untidy in documents and posts, and are easy to mistype.

The problem statement for this project was to design and develop a web-based URL shortener that converts a long URL into a short, unique link, stores the mapping persistently, and redirects anyone who opens the short link to the original website.

The key objectives of the project were:

- Provide a simple, user-friendly interface to enter a URL and obtain a short link.
- Validate the input and accept only supported protocols such as HTTP and HTTPS.
- Generate a unique short code for every URL and store the mapping in a SQLite database.
- Redirect the short link to the original URL and record the number of clicks.
- Return a proper not-found response for short codes that do not exist.
- Test, document and publish the project on GitHub.

4 Existing and Proposed solution

Existing solutions: Public services such as Bitly and TinyURL already shorten links reliably. However, they depend on third-party servers, advanced features such as analytics or custom domains often require sign-up or paid plans, the user has no control over how links are stored, and the internal working is hidden, which limits their learning value.

Proposed solution: A lightweight, self-hosted URL shortener built with Python and Flask. The user enters a URL on a web page; the application validates it, generates a unique short code, saves the mapping in SQLite through Flask-SQLAlchemy and displays the short link, which can be copied and shared. Opening the short link looks up the code, increments the click counter and redirects the browser to the original URL. Unknown codes return a not-found response.

Value addition: The application runs locally without a separate database server, tracks clicks for every link, offers recently generated links and a copy-link button, keeps routes, models and templates cleanly separated, and can easily be extended. The complete source code and documentation are provided for learning and reuse.

Technologies used in the project are summarised below.

Technology	Purpose
Python	Core application logic
Flask	Web framework: routes, HTTP requests, user input and redirection
Flask-SQLAlchemy	ORM layer to read and write the SQLite database
SQLite	Lightweight, persistent storage of URLs, short codes and click counts
HTML	Structure of the user interface
CSS	Appearance, layout and readability of the interface
Command Prompt / Git / GitHub	Running the app locally, version control and project submission

4.1 Code submission (Github link)

https://github.com/Adhiind/upskillcampus

4.2 Report submission (Github link)

https://github.com/Adhiind/upskillcampus/blob/main/URLShortener_Adhithiyaa_USC_UCT.pdf

5 Proposed Design/ Model

The system follows a simple client–server design. The browser sends HTTP requests to the Flask application, which validates input, interacts with the SQLite database through Flask-SQLAlchemy, and returns either an HTML page or an HTTP redirect. There are two main flows: creating a short link and opening a short link.

5.1 High Level Diagram (if applicable)

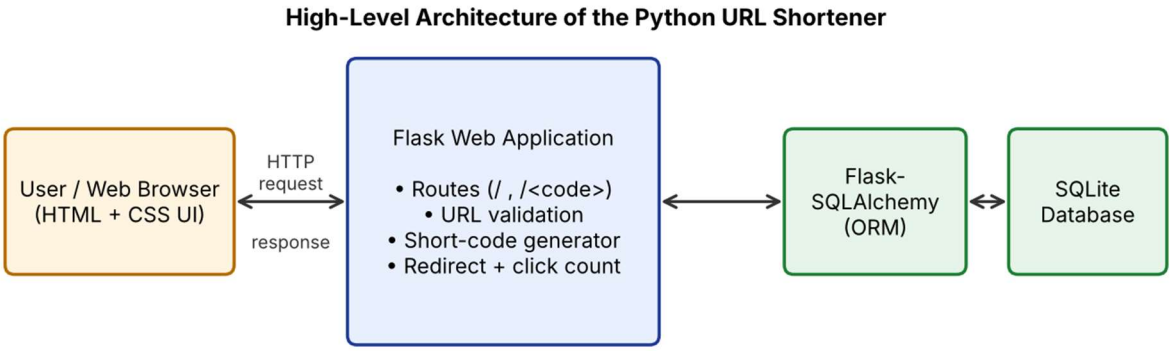


Figure 1: HIGH LEVEL DIAGRAM OF THE SYSTEM

The browser (HTML and CSS interface) talks only to Flask. Flask handles routing, URL validation, short-code generation, redirection and click counting, and uses Flask-SQLAlchemy to store and retrieve records from SQLite.

5.2 Low Level Diagram (if applicable)

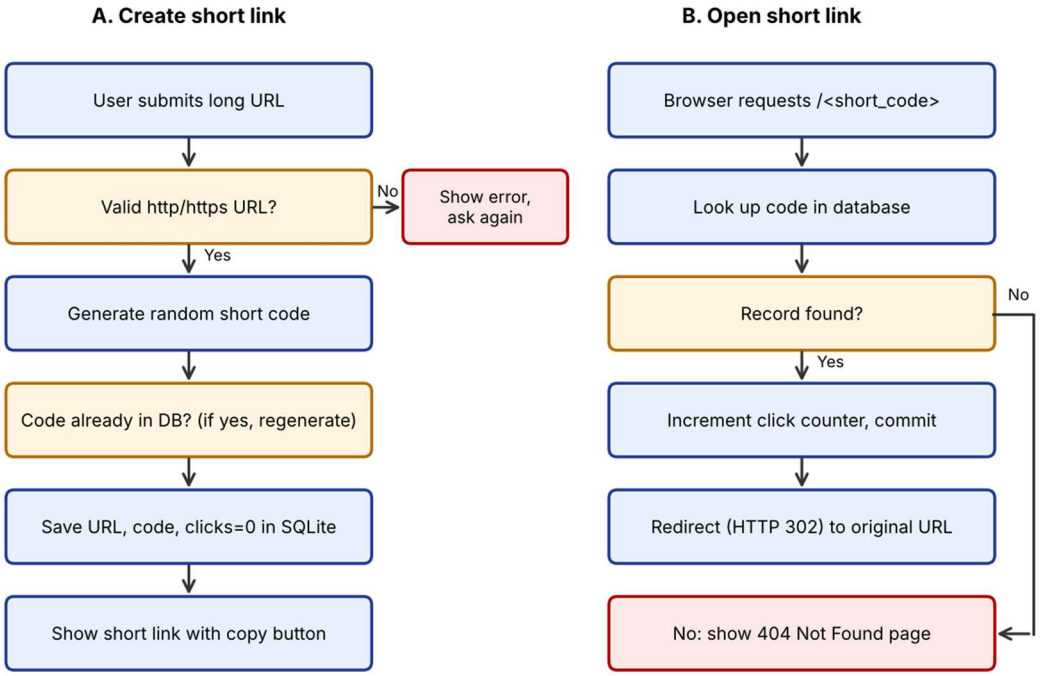


Figure 2: FLOW OF SHORT-LINK CREATION AND REDIRECTION

The database holds one table that maps each short code to its original URL. Field names may differ slightly in the source code.

Field	Type	Description
id	Integer (primary key)	Unique record identifier
original_url	String	The long URL entered by the user
short_code	String (unique)	Generated code that identifies the short link
clicks	Integer (default 0)	Number of times the short link has been opened

5.3 Interfaces (if applicable)

The application exposes a web interface and two HTTP routes. The home page contains a URL input box, a Shorten button, the generated short link with a copy-link option, and a list of recently generated links. Communication uses HTTP/HTTPS, and redirection is performed with an HTTP redirect response.

Route	Method	Function
/	GET, POST	Show the form; validate and shorten the submitted URL; display the short link
/<short_code>	GET	Find the original URL, increment the click count and redirect; not-found response if the code does not exist

6 Performance Test

The application is intended for educational and demonstration use, but it was still examined against practical constraints to make sure it behaves correctly and reliably.

Constraints and how the design handles them:

- **Input validity:** only well-formed HTTP/HTTPS URLs are accepted; empty, malformed or unsupported inputs are rejected before anything is stored.
- **Uniqueness:** every short code must be unique; the code is checked against the database and regenerated if it already exists.
- **Persistence:** mappings and click counts are stored in SQLite, so they survive application restarts.
- **Speed and memory:** the application is lightweight; lookups by short code are simple single-record queries, so response time is short for small data sets.
- **Scalability and concurrency:** SQLite and the Flask development server suit small-scale use; heavy traffic would need a production server and a client-server database such as PostgreSQL.
- **Security:** the current version has no authentication or rate limiting; these are recommended before any public deployment.

Test results are summarised in the sections below.

6.1 Test Plan/ Test Cases

The test plan covers input validation, short-code generation, redirection, click counting, error handling and persistence.

ID	Test case	Expected result

ID	Test case	Expected result
TC1	Submit a valid HTTPS URL	Unique short link is generated and displayed
TC2	Submit a valid HTTP URL	Unique short link is generated and displayed
TC3	Submit an empty input	Validation message shown; nothing is saved
TC4	Submit an unsupported protocol (e.g. ftp://example.com)	Input rejected with an error message
TC5	Submit malformed text (e.g. "hello world")	Input rejected with an error message
TC6	Open a generated short link	Browser is redirected to the original URL
TC7	Open the same short link several times	Click count increases by one on every visit
TC8	Open a short code that does not exist	Not-found response is shown; no redirect
TC9	Restart the application and open an earlier short link	Link still redirects (data persisted in SQLite)
TC10	Use the copy-link feature	Short link is copied for sharing

6.2 Test Procedure

The application was tested locally as follows:

- Install the required libraries (Flask and Flask-SQLAlchemy) using pip in the Command Prompt.
- Start the application by running the main Flask file (for example, python app.py).
- Open the local address shown in the console (typically http://127.0.0.1:5000/) in a web browser.
- Execute each test case TC1 to TC10 and compare the behaviour with the expected result.
- Check the click count and the stored records to confirm that the database is updated correctly.

6.3 Performance Outcome

The application meets its functional goals: valid URLs are converted into unique short links, invalid input is rejected, short links redirect to the original websites, click counts are updated on every visit, and unknown short codes produce a not-found response instead of an incorrect redirect. Data is retained between runs because it is stored in SQLite.

For small-scale local use, response time and resource usage are minimal. Limits on concurrency, security and scale are noted above and addressed in the future work scope.

7 My learnings

This project gave me a practical understanding of how a complete web application works from request to response, and the following learnings will support my career growth in software development:

- Web development with Python and Flask: routing, handling HTTP requests, form input and redirection.
- Database handling with Flask-SQLAlchemy and SQLite: defining models, storing, querying and updating records.
- URL validation, unique identifier generation, click tracking and error handling.
- Front-end basics with HTML and CSS to build a clean and readable user interface.
- Project practices: organising files, managing dependencies, testing, technical documentation and GitHub version control.
- Professional skills: planning work against a deadline, debugging patiently and presenting a project clearly.

8 Future work scope

Due to time limitations, the project was kept simple. It can be enhanced in the future with:

- Custom short links (user-chosen aliases).
- User authentication so each user can manage their own links.
- Link expiration dates.
- Detailed analytics such as visit time, device, browser and referrer.
- QR-code generation for every short link.
- Rate limiting and additional security measures, such as checking for malicious URLs.
- Cloud deployment with a production web server and a client-server database.